

Почему BIGSERIAL почти всегда лучше UUID для первичных ключей, даже в 2025 году

В последние десять лет стало модным использовать UUID как универсальный идентификатор. Их рекомендуют в блогах, курсах и репозиториях — будто это “единственно правильный” формат ключа. Однако в реальных производственных системах, особенно на PostgreSQL, UUID часто оказываются перегруженным и неоправданным выбором. Давайте разберёмся, почему.

Миф №1: «UUID — это обязательно, ведь у нас микросервисная архитектура!»

Аргумент звучит убедительно: множество сервисов создают записи независимо, значит нужен механизм глобальной уникальности.

Но реальность гораздо проще:

1. Глобальная уникальность обычному PK почти никогда не требуется.

Микросервисы обмениваются данными через API или очереди, а не напрямую через первичные ключи таблиц. Внешний “бизнес-ID” и внутренний PK — разные вещи.

2. Существуют более дешёвые альтернативы UUID:

- Snowflake-алгоритмы,
- ULID/KSUID (уникальные, сортируемые, компактные),
- распределённые последовательности PostgreSQL.

3. Если глобальный уникальный ID всё же нужен, его не обязательно делать PK.

Можно хранить его как отдельное поле, а саму таблицу индексировать компактным BIGINT.

Итого: «глобальная уникальность» — это аргумент за архитектуру, а не аргумент за UUID.

Миф №2: «UUID безопаснее, их не угадаешь»

Да, последовательный BIGINT может косвенно показать количество записей в таблице. Но это редко представляет реальную угрозу.

Если идентификатор уходит наружу пользователю — используйте:

- ULID,
- hashid,
- slug.

Внутренний PK при этом остаётся простым и быстрым BIGINT.

Большинство систем платят производительностью, RAM и диском за “безопасность”, которая им на самом деле не нужна.

Производительность: где UUID действительно проигрывает

1. Размер индекса

- UUID = **16 байт**
- BIGINT = **8 байт**

Чем больше индекс — тем хуже кэширование, тем чаще диск, тем медленнее работа.

2. Локальность в B-Tree

UUIDv4 распределён случайно → страницы дерева постоянно фрагментируются.

BIGINT растёт монотонно → вставка всегда идёт в “хвост” индекса → минимальная фрагментация и максимальная пропускная способность.

3. Массовые вставки

В нагрузочных тестах (pgbench) BIGINT стабильно показывает:

- **20–50% более высокую скорость INSERT,**
- меньше блокировок и конфликтов,
- более стабильное поведение под нагрузкой.

4. Стоимость владения

Большие индексы → больше памяти → больше репликации → больше нагрузки на VACUUM → больше денег.

В 2025 году это особенно заметно: объёмы растут быстрее, чем бюджеты.

Когда UUID действительно нужны

UUID оправдан, если:

- система работает оффлайн и должна генерировать ID локально,
- нет центрального хранилища ID,
- требуется публичный ID, который нельзя подобрать,
- применяется event sourcing и immutability-лог.

Это **специализированные** сценарии. Большинству CRUD-сервисов UUID попросту не нужен.

Вывод

UUID — мощный, но далеко не универсальный инструмент. Его преимущества часто переоценены, а недостатки в производительности и сложности — недооценены.

Для подавляющего большинства задач гораздо рациональнее использовать:

- **BIGSERIAL / BIGINT в качестве PK,**
- при необходимости — отдельный глобальный публичный идентификатор (ULID/KSUID/hashid).

Так вы получите:

- меньшие индексы,
- более потоковый и предсказуемый рост таблиц,
- более быструю вставку и выборку,
- более дешёвое обслуживание БД.

И именно поэтому — даже в 2025 году — **BIGSERIAL остаётся лучшим выбором первичного ключа в PostgreSQL для большинства систем.**